

Research Matching Chatbot System

Comprehensive Technical Architecture, Workflow & System Explanation Document

Executive Summary: The Research Matching Chatbot is an AI-driven, multi-agent academic platform built using **LangGraph**, **Google Gemini 2.5 Flash LLM**, and **Chroma Vector DB**. It acts as a dynamic bridge between students and academic faculty members, providing tailored faculty recommendations via Retrieval-Augmented Generation (RAG), automated research project ideation, interdisciplinary faculty collaboration analysis, live web research trend lookup, and professor-oriented strategic research synthesis.

1. System Overview & Operational Roles

The platform operates under two primary user modes tailored for academic ecosystems:

User Role	Core Capabilities	Underlying Mechanism
Student Mode	- Faculty RAG Matching - Project Ideation - Collaboration Discovery - Latest Research Web Search	LangGraph State Router (4 routes: faculty_rag, project, collaboration, web_search) connected to Chroma DB and Tavily Search.
Professor Mode	- Current Research Trends - Recent Advancements - Future Research Directions	Direct structured prompt synthesis via Gemini 2.5 Flash delivering structured, high-level research analysis.

2. LangGraph Stateful Workflow Engine

At the core of the Student Mode execution is a **LangGraph StateGraph** router (defined in graph/student_graph.py). Unlike basic chain-based LLM calls, LangGraph maintains a state dictionary (StudentState) and evaluates intent dynamically.

State Schema Definition (StudentState):

```
class StudentState(TypedDict):
    query: str          # Original prompt entered by the student
    route: str          # Target route determined by classifier
    response: str       # Generated final text response or synthesis
    matches: list       # List of retrieved matching faculty dictionaries
    context: str        # Formatted text context of top faculty profiles
```

Graph Nodes & Routing Logic:

- **START → Classify Node:** Analyzes incoming query string keywords to set route flag.
- **Conditional Edge (route_query):** Directs state execution based on route classification:
 1. collaboration → Executes collaboration_node (Finds top 2 faculty pairs).

- 2. web_search → Executes web_search_node (Invokes Tavily API + Gemini synthesis).
- 3. project or faculty_rag → Executes faculty_retrieve_node (Chroma DB similarity search).

Intent Classifier Keyword Rule	Target Route	Execution Flow & Output
Contains 'collaboration'	collaboration	Retrieves top 2 faculty profiles and crafts interdisciplinary synergy reasoning.
Contains 'latest' or 'trend'	web_search	Queries Tavily search API for real-time web results and synthesizes summary.
Contains 'project'	project	Retrieves faculty matches and generates 3 detailed project ideas with difficulty ratings.
Default query	faculty_rag	Performs Chroma DB RAG search and outputs matched faculty with similarity score.

3. Vector Database & RAG Infrastructure

Faculty profiles stored in data/faculty_profiles/faculty.json are converted into vector embeddings using Google's **gemini-embedding-001** model and persisted locally in **Chroma DB**.

Key Engineering Architectural Highlights:

- **SHA-256 Cache Syncing:** A .data_hash file stores the SHA-256 hash of faculty.json. Database re-indexing is triggered automatically only when data modification is detected.
- **Strict Publication Filtering Rule:** Only faculty members with listed publications are indexed into the vector store, ensuring high accuracy academic retrieval.
- **Similarity Score Normalization:** Distance scores returned by Chroma vector similarity searches are converted into percentage match metrics using the formula:
$$\text{Cosine Similarity \%} = \max(0.0, 1.0 - (\text{Score} / 2.0)) * 100$$
- **Robust Hybrid Fallback Engine:** If vector DB loading or embedding API is unavailable, retriever.py gracefully falls back to an in-memory tokenized keyword matching engine across faculty profiles and publications.

4. Tool Modules & Agent Extensions

- **tools/load_data.py:** Handles Chroma DB initialization, SHA-256 hash tracking, JSON document transformation, and embedding indexing with gemini-embedding-001.
- **tools/retriever.py:** Performs vector similarity search against Chroma DB with exact publication keyword boost and fallback tokenized matching.
- **tools/project_suggester.py:** Uses Gemini 2.5 Flash to generate 3 customized project ideas, difficulty levels, required skills, and expected outcomes based on matched faculty context.
- **tools/collaboration.py:** Analyzes research intersections between top matching professors to propose collaborative interdisciplinary research initiatives.

- **tools/tavily_search.py**: Integrates Tavily Web Search API to fetch live external web context for queries regarding breaking research news and latest industry trends.
- **research_agents/professor_agent.py**: Dedicated agent module for professors that synthesizes current trends, recent advances, and future research directions.

5. User Interface & Streamlit Web Layer

The application features a modern Streamlit user interface (streamlit_app.py) designed for usability:

- **Dual-Role Landing Screen**: Interactive selection between *Student Mode* and *Professor Mode*.
- **Dynamic Sidebar Controls**: Provides role indicator, role switcher, chat clearing, quick examples, and a manual  *Reload / Sync Data* button to force vector index rebuilds.
- **Formatted Match Output**: Displays faculty cards with percentage match indicators, department, mobile contact info, research topics, and publications.
- **Quick Prompt Chips**: Enables single-click prompt insertion for standard student and professor queries.

6. Project Directory Map

```
sai2/
├── app.py                # Command-line interface (CLI) launcher
├── streamlit_app.py      # Web GUI application (Streamlit)
├── config.py             # Environment & API Key loader (.env)
├── requirements.txt      # Dependencies list
├── data/
│   ├── faculty_profiles/
│   │   └── faculty.json  # Structured JSON database of faculty members
│   └── graph/
│       ├── student_graph.py  # LangGraph state graph definition & router
│       └── research_agents/
│           ├── student_agent.py  # Student query handler & graph runner
│           └── professor_agent.py # Professor research query processor
├── tools/
│   ├── load_data.py        # Data preprocessor & Chroma DB loader
│   ├── retriever.py        # Hybrid RAG retriever engine
│   ├── collaboration.py    # Faculty collaboration match tool
│   ├── project_suggester.py # Gemini project ideation engine
│   ├── rag.py              # Standalone RAG CLI demo script
│   ├── tavily_search.py    # Tavily live web search API client
│   └── chroma_db/          # Local persistent Chroma vector store
```

7. Setup, Environment & Execution Guide

1. Environment Configuration (.env):

```
GOOGLE_API_KEY=your_gemini_api_key_here
TAVILY_API_KEY=your_tavily_api_key_here
```

2. Run Streamlit Application:

```
streamlit run streamlit_app.py
```

3. Run Command-Line Chatbot Interface:

```
py app.py
```

4. Rebuild Vector Index manually:

```
py tools/load_data.py
```